KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 2 Laboratory Sheet

Constraint Propagation: Consistency, AC-3, and Measuring Propagation Strength

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours

Step 11 steps · 5 checkpoints

Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying `ortools`, no new package this week), a text editor

1 2 3 ✓ CP1 4 5 ✓ CP2 6 7 ✓ CP3 8 9 ✓ CP4 10 11 ✓ CP5

Learning phases: 1 · Pre-instruction | 2 · Content | 3 · Participation | 4 · Assessment | 5 · Follow-through

Why It Matters 1 · Pre-instruction

Every solver call you made last week either found a solution or did not, with nothing visible happening in between. Today you open that gap and watch what actually happens *before* a solver ever guesses a variable's value: values get thrown out for provably never being able to appear in any solution, by hand first, then by a worklist algorithm you trace by hand and then run as real code, then by five global constraints each hiding a nontrivial algorithm behind a one-line call. You close by building two CP-SAT models of the identical relation and reading the solver's own statistics to see, in numbers you capture yourself, exactly how much stronger one model's filtering really is.

Prior Knowledge 1 · Pre-instruction

This sheet exercises all four units of this week's chapter: node, arc and generalized arc consistency (Unit 1), the AC-3 algorithm and its fixpoint (Unit 2), what five global constraints propagate (Unit 3), and measuring propagation strength empirically on two CP-SAT models of the same problem (Unit 4). Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Confirming the Shared Environment

Step 1: **Confirm this week needs no new package**

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` active, you will print the installed `ortools` version to confirm the same package Week 1 installed is still there — this week is CP-SAT only, no new package install needed, and `ac3.py` needs no package at all.

Why? Every `ciktikutusu` box in this sheet is a real, captured run at this exact `ortools` version; confirming the version before you start is what makes those boxes reproducible on your own machine.

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ python -c "import ortools; print(ortools.__version__)"
```

Expected result: The command prints `9.14.6206`, the exact version Week 1 pinned for the whole course.

Expected Output

```
9.14.6206
```

Tip

If this command raises `ModuleNotFoundError`, your virtual environment is not active in this terminal — re-run `source .venv/bin/activate` from inside `kod_ornekleri` before continuing.

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead.

2 Unit 1 — Local Consistency: Node, Arc and Generalized Arc Consistency

(See Course Notes Week 2, Unit 1 — Local Consistency: Node, Arc and Generalized Arc Consistency.)

Step 2: **Enforce node consistency on a new booking variable**

What will you do? A shared electron microscope has a booking variable `Session`, domain {8am, 11am, 3pm}, under the unary rule "the microscope is recalibrated every day at 3pm." On paper, decide whether `Session` is node-consistent, and if not, state the domain that remains once node consistency is enforced.

Why? This is Unit 1's own `Slot` toy (a variable checked purely against its own unary rule), applied here to a fresh scenario you have not seen worked out yet — writing out the check yourself on a new instance is what turns following the definition into applying it.

Expected result: `Session` is not node-consistent as given: the value 3pm violates the unary rule — the microscope is being recalibrated then, so no booking can use it — while 8am and 11am both satisfy the rule. Removing 3pm leaves the node-consistent domain {8am, 11am} — nothing about any other variable was consulted to reach this, exactly as in Unit 1's own `Slot` example.

Step 3: Classify a three-group booking CSP by consistency level

What will you do? After the previous step, only two microscope sessions remain: {8am, 11am}. Three research groups, `G1`, `G2`, `G3`, each need exclusive use of one session, under `AllDifferent(G1, G2, G3)`. (a) Check arc consistency pairwise: does every value of every group have a supporting value in each other single group's domain, checked one pair at a time? (b) Check generalized arc consistency on the whole `AllDifferent` constraint directly: can every value be extended to a complete, jointly satisfying assignment of all three groups? State whether this booking CSP is solvable at all.

Why? This is Unit 1's three-variable, two-value pigeonhole toy, renamed onto a lab-resource-booking scenario built directly out of Step 1's own domain — reasoning through the same pigeonhole argument on numbers you derived yourself, rather than numbers handed to you already reduced, is what confirms you can apply Unit 1's ladder end to end, not just recall its conclusion.

Expected result: (a) Yes, pairwise arc-consistent: for any pair, e.g. `G1` = 8am has a supporting value `G2` = 11am, and symmetrically for every other pair and value — no domain is reduced by checking pairs one at a time. (b) No: with only 2 sessions available for 3 groups that must all differ, the pigeonhole principle rules out every joint assignment, so *no* value in *any* domain has a valid GAC support against the whole `AllDifferent` constraint. This booking CSP is infeasible, and a GAC-aware check catches that immediately, exactly as Unit 1's A, B, C example did — while the pairwise check in (a) never finds the wipeout, because each pair alone is still satisfiable.

Tip

Hold onto this exact contrast: Unit 2's AC-3 algorithm, which the next section runs as real code, only ever checks *binary* arc consistency. Run on this booking CSP's pairwise decomposition, AC-3 would report "arc-consistent: true" too, just like your part (a) above — it has no way to see the whole-group pigeonhole argument that made part (b) say no. Unit 3 names the algorithm (Régin's matching) that lets `AddAllDifferent` catch this automatically, on CSPs far larger than three variables.

Checkpoint — verify before moving on

You have enforced node consistency on a fresh booking variable by hand, then classified a three-group version of that same booking scenario at both the arc-consistency and GAC levels, reaching the same "pairwise looks fine, GAC proves infeasible" conclusion Unit 1's own worked example reached, on numbers of your own. You are ready to see the algorithm that actually *computes* arc consistency, rather than just checking it by hand.

3 Unit 2 — AC-3 and the Propagation Fixpoint

(See Course Notes Week 2, Unit 2 — AC-3 and the Propagation Fixpoint.)

Domain propagation on the chain CSP $X < Y < Z$

Before propagation	
Var	**Domain**
X	{1, 2, 3}
Y	{1, 2, 3}
Z	{1, 2, 3}

AC-3 →

After propagation (fixpoint)	
Var	**Domain**
X	{**1**, ~~2~~, ~~3~~}
Y	{~~1~~, **2**, ~~3~~}
Z	{~~1~~, ~~2~~, **3**}

Chain CSP $X < Y < Z$: arc consistency alone reaches the unique solution — see Week 5 for when this always happens.

Step 4: **Run the incomplete ac3_chain_lab.py and find the missing filtering**

What will you do? A starter file, `kod_ornekleri/hafta02/lab/ac3_chain_lab.py`, reuses `modeller/ac3.py`'s `revise`/`ac3` engine unchanged on a new four-variable chain CSP, $W < X < Y < Z$, all domains starting at $\{1, 2, 3, 4\}$. Its `arcs` list is deliberately incomplete — only the forward direction of each constraint is queued, and the three reverse arcs are left as a `TODO`. Before running anything, work out by hand what the true fixpoint of this chain *should* be (there is exactly one way to place four distinct values on a strict chain of four variables over a domain of exactly four values). Then run the file exactly as given and compare its printed domains against your hand answer.

Why? Unit 2's own `ipucu` names re-queueing a neighbour's arcs as "the one detail students most often skip" — running a chain CSP whose initial worklist only ever queues one direction per constraint is what turns that warning from a sentence you read into a wrong answer you watch happen on your own screen.

The deliberately incomplete arcs list:

```
36 arcs = [("W", "X"), ("X", "Y"), ("Y", "Z")]
37 # TODO: this list only queues the forward direction of each
38 # constraint. Add the three missing reverse arcs -- ("X", "W"),
39 # ("Y", "X"), ("Z", "Y") -- the same way modeller/ac3.py's
40 # build_chain_csp() queues both directions of every constraint.
```

Terminal

```
$ cd hafta02
$ python lab/ac3_chain_lab.py
```

Expected result: By hand: a strict chain $W < X < Y < Z$ over domain $\{1, 2, 3, 4\}$ has exactly one satisfying assignment, $W = 1, X = 2, Y = 3, Z = 4$, so the true fixpoint should reduce every domain to that single value. The buggy run instead reports `Arc-consistent: True` while leaving `Z` completely untouched — Z's domain never shrinks at all, because no arc in this incomplete list ever filters Z against Y.

Expected Output

```
Domains before AC-3: W=[1, 2, 3, 4], X=[1, 2, 3, 4], Y=[1, 2, 3, 4], Z=[1, 2, 3, 4]
Arc-consistent: True
Domains after AC-3: W=[1], X=[1, 2], Y=[1, 2, 3], Z=[1, 2, 3, 4]
```

Common Pitfall

A printed `Arc-consistent: True` looks like a clean pass, which makes it tempting to trust the domains next to it without checking them against your own hand-worked fixpoint. Here that trust would be misplaced: `True` only means "no `Revise` call in *this restricted arc list* ever found an unsupported value," not "the CSP is actually fully arc-consistent." A missing reverse arc does not raise an error; it silently leaves values behind that a complete worklist would have removed.

Step 5: **Add the missing reverse arcs and verify the true fixpoint**

What will you do? In `lab/ac3_chain_lab.py`, where the `TODO` comment says to, add the three missing reverse arcs — `("X","W")`, `("Y","X")`, `("Z","Y")` — to the `arcs` list, the same way `modeller/ac3.py`'s own `build_chain_csp()` queues both directions of every constraint. Then re-run the script.

Why? This is Unit 2's own worklist idea, applied by your own hand rather than only traced on someone else's chain: adding exactly the arcs the previous step's `ortakhata` named as missing is what turns explaining why re-queueing matters into fixing a worklist and watching the correct fixpoint appear.

Terminal

```
$ python lab/ac3_chain_lab.py
```

Expected result: Every domain collapses to the single value your hand-worked chain predicted: $W = \{1\}, X = \{2\}, Y = \{3\}, Z = \{4\}$ — this time including Z, which the buggy run above left completely unfiltered.

Expected Output

```
Domains before AC-3: W=[1, 2, 3, 4], X=[1, 2, 3, 4], Y=[1, 2, 3, 4], Z=[1, 2, 3, 4]
Arc-consistent: True
Domains after AC-3: W=[1], X=[2], Y=[3], Z=[4]
```

Tip

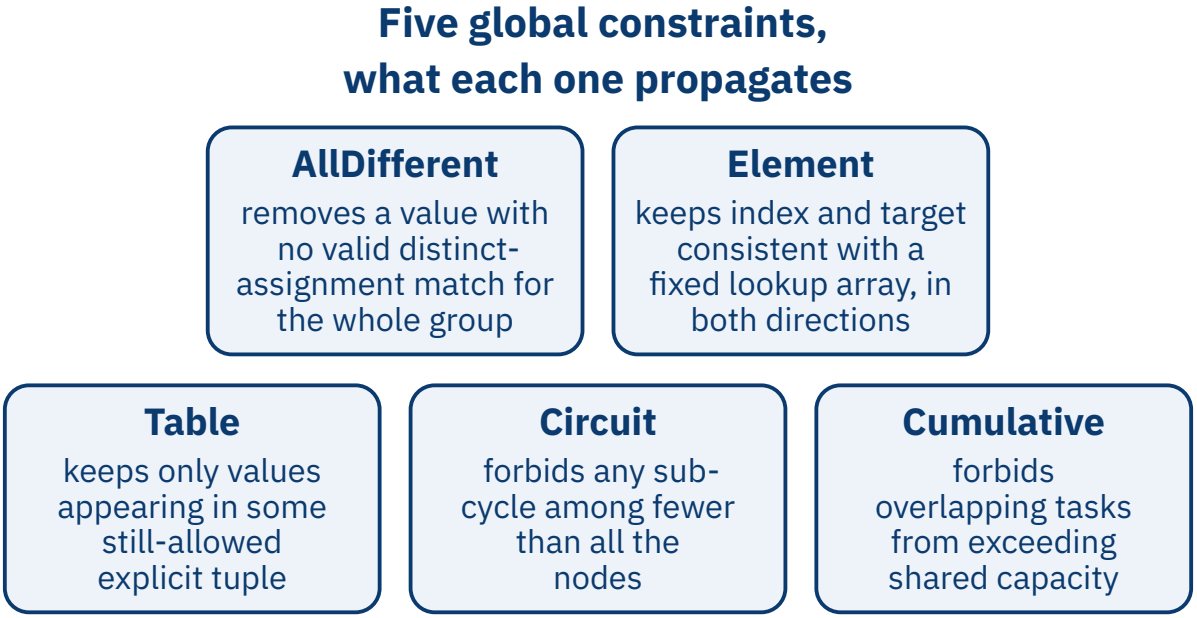
If your domains still do not fully collapse, check the arc list's *directions*, not just its length: `("X","W")` filters X against W, which is not the same arc as `("W","X")` that was already there — AC-3 needs both, for every constraint.

Checkpoint — verify before moving on

You have watched a deliberately incomplete worklist report `Arc-consistent: True` while silently leaving a variable's domain completely unfiltered, then added exactly the missing reverse arcs and watched every domain collapse to the chain's unique solution. You now have direct, hands-on evidence for why AC-3 re-queues both directions of every arc, not just a definition you can recite.

4 Unit 3 — Global Constraints: What They Propagate

(See Course Notes Week 2, Unit 3 — Global Constraints: What They Propagate.)



Step 6: **Run all four global-constraint demos in one pass**

What will you do? You will run `global_constraints_demo.py`, which prints all four of Unit 3's remaining toy demonstrations in sequence — `Element`, `Table`, `Circuit`, `Cumulative` (`AllDifferent` is already covered by Unit 4's N-queens models below) — and check each block against the matching subsection of the chapter.

Why? Unit 3 walks through each demo's code and captured output one at a time; running the whole file once here is what confirms all four demos are, together, exactly the real code behind that prose, not four separate coincidences.

Terminal

```
$ python modeller/global_constraints_demo.py
```

Expected result: Four blocks, printed in order: the `Element` demo narrowing `index` to $\{1, 2\}$ once `target >= 20` is added; the `Table` demo narrowing `color2` to `['G', 'B']` once `color1` is fixed to R; the `Circuit` demo's single 4-node cycle; and the `Cumulative` demo's three start times, none exceeding the resource's capacity of 4.

Expected Output

```
Element demo, menu prices=[10, 25, 40, 15]
index domain before target >= 20: [0, 1, 2, 3]
index domain after target >= 20: [1, 2]
Table demo, allowed (color1, color2) pairs: [(0, 1), (0, 2), (1, 0), (1, 2),
(2, 0), (2, 1)]
color1 fixed to R; color2 narrows to ['G', 'B']
Circuit demo (4 nodes): visiting order [0, 2, 1, 3] -> back to 0
Cumulative demo, durations=[3, 2, 2], demands=[2, 3, 2], capacity=4:
start times = [2, 0, 2]
```

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta02` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta02`.

Step 7: **Explain the Cumulative demo's propagation in your own words**

What will you do? Using only the `Cumulative` block of the previous step's real output — durations $[3, 2, 2]$, demands $[2, 3, 2]$, capacity 4, start times $[2, 0, 2]$ — write, in your own words, which start-time combinations `Cumulative` ruled out and on what basis, checking by hand that no instant's total demand exceeds the capacity of 4.

Why? Unit 3 defines what `Cumulative` propagates abstractly, then checks one specific captured solution by hand; redoing that same hand check yourself, on the real output you just produced, is what confirms you can read a `Cumulative` result rather than only follow the chapter's own worked check.

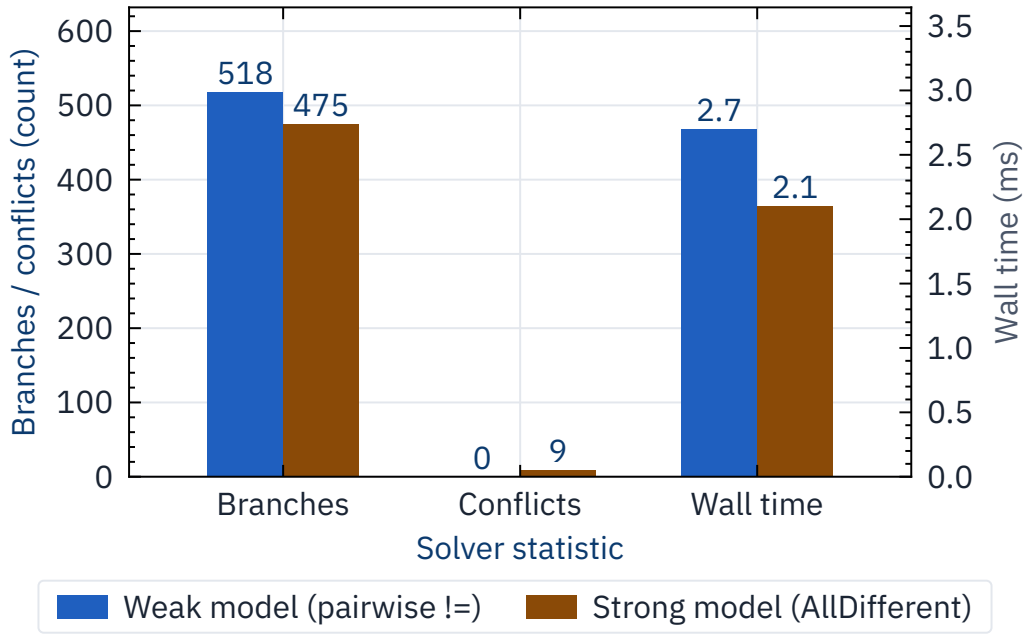
Expected result: Task 1 (demand 3) occupies $[0, 2)$ alone. Tasks 0 and 2 (demands 2 and 2 each) both occupy $[2, 4)$, together demanding $2 + 2 = 4$ — exactly at, never past, the capacity of 4. `Cumulative` ruled out, before any start time was fixed, every combination that would have pushed some instant's total demand above 4 — for example, all three tasks overlapping at once would demand $2 + 3 + 2 = 7$, far past capacity, and that combination was never even offered as a candidate start-time assignment to check.

Checkpoint — verify before moving on

You have run every one of Unit 3's four remaining global-constraint demos from real, captured output and re-derived the `Cumulative` demo's capacity check by hand yourself. Together with Unit 1's `AllDifferent` pigeonhole argument, you have now worked through what every one of this chapter's five global constraints propagates and why.

5 Unit 4 — Measuring Propagation Strength: Weak vs. Strong Models

(See Course Notes Week 2, Unit 4 — Measuring Propagation Strength: Weak vs. Strong Models.)



8-queens, same problem, two models — real CP-SAT statistics from one run.

Step 8: Run the N-queens propagation comparison at N=8 and N=20

What will you do? You will run `nqueens_propagation_compare.py`, which solves both the strong (`AllDifferent`) and weak (pairwise `!=`) N-queens models from `nqueens_models.py` at $N = 8$ and then again at $N = 20$, and record `NumBranches`, `NumConflicts` and `WallTime` for all four solves from your own terminal output.

Why? This is Unit 4's central empirical claim, run on your own machine rather than only read from the chapter's prose — reading these four real solves yourself is what turns "propagation strength can be measured" from an assertion into something you watched a solver actually report.

Terminal

```
$ python modeller/nqueens_propagation_compare.py
```

Expected result: At $N = 8$, the strong model needs fewer branches and less wall time than the weak model, but *more* raw conflicts — report this exactly as printed, not adjusted to fit the expected direction. At $N = 20$, the same branches/wall-time pattern repeats, now with a clearly larger wall-time gap, while conflicts again move the other way.

Expected Output

```
N=8 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=475 conflicts=9 wall_time=0.0021s
weak (pairwise !=): status=OPTIMAL branches=518 conflicts=0 wall_time=0.0027s
N=20 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=3312 conflicts=51 wall_time=0.0090s
weak (pairwise !=): status=OPTIMAL branches=3940 conflicts=26 wall_time=0.0201s
```

Caution

Your own `wall_time` numbers will almost certainly differ from the ones above — wall-clock time depends on your machine, not on the model. `branches` and `conflicts`, by contrast, come from a fixed `random_seed` and a single search worker, so those two counts should match the boxes above exactly on any machine running the pinned `ortools` version.

Step 9: Interpret the larger instance against the presolve caveat

What will you do? Unit 4 warns that CP-SAT's own presolve can narrow the measured gap between the weak and strong model at small N. Using only the two blocks of real output from the previous step, state in one or two sentences whether moving from $N = 8$ to $N = 20$ made the strong model's advantage on branches and wall time larger, smaller, or unchanged in relative terms, and whether that is consistent with Unit 4's caveat.

Why? This is Unit 4's own honest-caveat reasoning, applied by you to numbers you just produced rather than only read in the chapter's prose — deciding for yourself whether a larger instance strengthens or weakens the measured gap is what H5 asks: connecting an empirical result back to the argument that predicted it.

Expected result: At $N = 8$ the strong model's wall-time advantage is modest (0.0021s against 0.0027s, roughly 22% faster); at $N = 20$ it is substantially larger in relative terms (0.0090s against 0.0201s, more than twice as fast). This is consistent with Unit 4's caveat: presolve's ability to tighten a pairwise decomposition matters relatively more when the decomposition is small, so a noticeably larger instance is exactly where the strong model's structural advantage is expected to show up more clearly — which is what these two real runs, taken together, show.

Checkpoint — verify before moving on
You have run and recorded four real solves — strong and weak, at $N = 8$ and $N = 20$ — and reasoned about what a larger instance revealed about presolve's effect on the measured gap. You are ready to build this same comparison yourself, on a problem this sheet has not already solved for you.

6 Capstone — Predict, Then Verify, on a New CSP

Step 10: Predict the propagation-strength winner before running anything

What will you do? Six review sessions each need one of six one-hour grading slots, no two sessions sharing a slot — a pure "everyone differs from everyone" relation over 6 variables and a domain of exactly 6 values, unlike Week 1's map-coloring adjacency (see this chapter's Check Yourself Q4). Before writing or running any code, use Unit 1's pigeonhole argument and Unit 3's `AllDifferent`/Régin explanation to predict, in writing: should the strong (`AllDifferent`) or the weak (pairwise `!=`) model need fewer branches here, and why might the gap be even more pronounced than N-queens' $N = 8$ gap was?
Why? This is H5 in its most direct form: stating a prediction from the consistency-level argument *before* seeing a single real number is what makes the next step's real run a genuine test of your reasoning, not a number you read and then explain after the fact.

Expected result: The strong model should need markedly fewer branches. Unlike N-queens (where three separate `AllDifferent` groups interact through shared row/diagonal structure), this instance is a single, isolated `AllDifferent` over exactly as many variables as available values — Régin's matching-based filtering can, in principle, detect on its own that every value must be used exactly once and propagate the entire assignment through consistency alone, needing little or no search. The weak pairwise decomposition has no such single-shot argument available to it and must instead discover the same conclusion by branching, so a substantially larger gap than N-queens' modest $N = 8$ gap is a reasonable prediction here.

Step 11: Complete exam_slots_capstone_lab.py and verify your prediction

What will you do? A starter file, `lab/exam_slots_capstone_lab.py`, already builds the strong model with a single `AddAllDifferent` call; the weak model's pairwise `!=` loop is left as a `TODO`. Complete it the same way `nqueens_weak_constraints` does for N-queens' column values in `modeller/nqueens_models.py`, then run the file and compare the real statistics against the prediction you wrote down in the previous step.
Why? This closes H5's loop for real: your own completed code, run on your own machine, either confirms or complicates the prediction you committed to on paper first — exactly the "argument, then evidence" pattern this entire chapter has been building toward.
The TODO you are completing:

```
36 # TODO: add one model.Add(sessions[i] != sessions[j]) constraint for
37 # every pair i < j, the same way nqueens_weak_constraints does for
38 # N-queens' column values in modeller/nqueens_models.py.
```

Terminal

```
$ python lab/exam_slots_capstone_lab.py
```

Expected result: The strong model solves with *zero* branches and zero conflicts — consistency alone, with no search at all, was enough to find and confirm a solution. The weak model needs real search. This gap is far more decisive than N-queens' $N = 8$ result, confirming the previous step's prediction: an isolated `AllDifferent` over exactly as many variables as values is close to the strongest case this chapter's argument can make.

Expected Output

```
N=6 exam-slot propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=0 conflicts=0 wall_time=0.0007s
weak (pairwise !=): status=OPTIMAL branches=71 conflicts=0 wall_time=0.0006s
```

Common Pitfall

Running the file *before* completing the `TODO` does not raise an error — an empty weak model has no constraints at all, so it reports 0 branches too, which looks like "no gap" rather than "not yet a real comparison." Always fill in the missing pairwise loop first; a weak model that never gained any constraints cannot be compared to anything.

Tip

If your own `branches`/`conflicts` numbers do not match the box above exactly, check that every pair `i < j` was covered exactly once by your loop, the same nested-range pattern `nqueens_weak_constraints` uses.

Checkpoint — verify before moving on

You have predicted, on paper and before running anything, which model should show stronger propagation on a new CSP, then completed and run that model yourself and confirmed the prediction against real solver statistics — the same H1/H3-to-H4 loop Unit 4 closed on N-queens, now closed by you, end to end, on a problem this sheet did not solve for you first.

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`
Running any `modeller/*.py` or `lab/*.py` script prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta02` before running any script
`ac3_chain_lab.py` still leaves `Z` unfiltered after your edit	The new arcs were appended in the wrong direction, e.g. `("W","Z")` instead of `("Z","Y")`	Re-check each new arc against the constraint it filters: `("Z","Y")` filters Z using Y's current domain, not the other way around
Your completed capstone script reports the same branch count for both models	The pairwise loop was added inside `build_strong_model` instead of `build_weak_model`, so both functions end up with identical constraints	Confirm the `TODO` loop sits inside `build_weak_model`, directly under the `exam_slots_weak_constraints` tag
The N-queens comparison script runs but never finishes	`num_workers` or `random_seed` was removed from `collect_solver_stats` while editing a copy of the file	Restore `modeller/*.py` from the course repository unedited; only files inside `lab/` are meant to be edited this week
`pytest -q` reports failures unrelated to your own edits	`modeller/` files were edited directly instead of the `lab/` starter files	Restore `modeller/*.py` from the course repository; only the two starter files inside `lab/` are meant to be edited this week

Transfer — Where Else? 5 · Follow-through

Every idea you exercised today keeps returning under new arithmetic: Week 7's shadow-price and reduced-cost updates and Week 15's unsat-core extraction both reuse this week's worklist/fixpoint shape, Week 6 and Week 7 repeat Unit 4's "same problem, two models, compare the real solver statistics" habit across MiniZinc's solver back-ends and against a Gurobi MIP formulation, and Week 11 and Week 14 return to `Cumulative` and `Circuit` by name, no longer as tiny toy demonstrations but as the backbone of real scheduling and routing models. More generally, the habit this sheet asked for five times — argue what a stronger check *should* catch, then run real code and read whatever it actually reports, including a result that does not fit the expected pattern perfectly — is this week's version of the single habit this course exists to build, now extended from "does a solution exist" to "how hard did the solver have to work to find it, and why."